

简单树论与树上技巧

Part II

wYYSZL

2026.8.10

本节课内容

- ① 生成树相关
- ② 树上差分
- ③ 树上启发式合并
- ④ 杂题

目录

- ① 生成树相关
- ② 树上差分
- ③ 树上启发式合并
- ④ 杂题

最小生成树

生成树一般针对无向连通图。

生成树

给定一张图，它的**生成子图**为保留所有点，保留部分边的图。

给定一张无向连通图，它的**生成树**是形态为树的生成子图。（连通，无环）

一张图的最小生成树指的是边权和最小的生成树。

求最小生成树通常有两种做法，分别是 Kruskal 算法和 Prim 算法。前者比较常用。

最小生成树求解 - Kruskal 算法

Kruskal 算法非常简单，它的核心思路是：从小到大考虑每一个边，如果能加进去，即加入后不成环，就加，否则跳过。

具体地，Kruskal 算法的流程如下：

1. 将所有边按权重**非递减**排序： $e_1, e_2, \dots, e_m$ 。
2. 初始化森林 $F = \emptyset$ （每个顶点自成一个连通分量）。
3. 依次扫描每条边 e_i ，若 e_i 加入后会形成环，即它的两端在 F 中相同的连通分量，则丢弃，否则将其加入 F 。判断是否已经连通可以使用并查集。
4. 重复直到 F 包含 $n - 1$ 条边（ $n = |V|$ ）。

总的时间复杂度 $\mathcal{O}(m \log m)$ 。

最小生成树求解 - Kruskal 算法

Kruskal 算法为什么是对的？这个证明有很多种，这里讲一个有趣的。

首先设每一条边的边权不同。这时 Kruskal 算法求出的边集确定，设为 F 。设 Kruskal 算法没有求出最小生成树，最小生成树边集是另一个（设为 E ）。

假设 F 中最小的不在 E 中的边为 $e = (u, v)$ 。我现在强行将 e 加入 E 中，就会成环。由于 Kruskal 算法会选择 e ，说明加入 e 时还不连通。 E 没有选择 e ，那么这个环中一定有一条边 l ，使得 l 不在 F 中。

又由于 e 的最小性， $dis_l > dis_e$ 。如果删去 E 中的 l 而换为 e ，则会获得更小的生成树，矛盾。

最小生成树求解 - Kruskal 算法

现在考虑有相同边权。设任意非最小生成树与最小生成树的权值之差的最小值 q 。

这时我们采用微扰法，为相同的边权加一个 $i \times eps$ ，其中 i 为边的编号，使得边权不同。这个 eps 远远小于 q 。

这时，我们已经证明，在这个新图中，Kruskal 算法可以求出最小生成树。那么，保留这条边，回到原图时，总的边权和会有变化，但不会退化到原图的非最小生成树中。

此外

上面的过程也顺便证明了，当边权两两不同时，最小生成树唯一。

最小生成树求解 - Prim 算法

Prim 算法也是贪心的做法，它的基本思想是，从一个点开始，不断地加点。

具体地流程如下：

首先任选一个点加入集合 S 。

1. 遍历所有没有加入 S 的点 x ，找到距离 S 最近的点。为了方便，记 $dist_x$ 为 x 到 S 的距离。
2. 将距离最近的 x 加入 S 中。
3. 用 x 更新 $dist$ 数组。

整个算法的复杂度是 $\mathcal{O}(n^2)$ 。类似 Dijkstra 算法，可以用堆优化，但是优化后复杂度为 $\mathcal{O}((n + m) \log n)$ ，不如 Kruskal。

因而 Prim 算法通常用于非常稠密的图（当 $m = \Theta(n^2)$ 时），并且使用朴素的 Prim。也类似于朴素的 Dijkstra 的适用范围。

例题 E

E - 【模板】最小生成树

给出一个无向图，求出最小生成树。

例题 E

E - 【模板】最小生成树

给出一个无向图，求出最小生成树。

模板题。

Kruskal 重构树

Kruskal 重构树是 Kruskal 算法过程的静态呈现。

我们结合构造过程对其进行定义：

初始将原图 n 个点视为 n 棵独立的树（根节点即自身）。随后将边按权值从小到大排序，依次扫描每条边 (u, v, w) ：

1. 查询 u 和 v 当前所在重构树的根节点（并查集维护）。
2. 若根不同，则新建一个节点 p ，令其点权 $val[p] = w$ 。
3. 将 u 和 v 所在的树根分别作为 p 的左右儿子，并将 p 设为新连通块的根。

算法结束时，得到一棵二叉树（或森林），其中叶子节点是原图点，内部节点是按边权升序创建的虚点。

直观上讲，就是在加边的时候，不只是加边，而是用一个有点权的点表示这条边，然后它作为两个连通块代表的父亲。

Kruskal 重构树

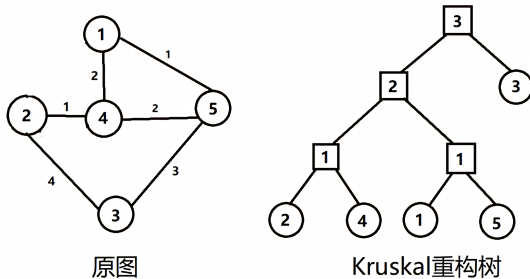


图: Kruskal 重构树例子

其中方块内的数字不是编号而是点权。

Kruskal 重构树

Kruskal 重构树有一些良好的性质。

性质

1. 点权从下到上单调不减。
2. 图中两个点之间的所有简单路径上最大边权的最小值 = 最小生成树上两个点之间的简单路径上的最大值 = Kruskal 重构树上两点之间的 LCA 的权值

性质 1 与性质 2 的第一个等号都可以通过 Kruskal 过程解释，性质 2 的第二个等号结合第一个性质可以证明。

有了这个性质，意味着一个点 x 出发，只走边权 $\leq val$ 的路，可以到达的点的集合，就是 x 的 $\leq val$ 祖先中，最浅的那个点的子树中的所有点。

例题 F

F - 归程

给定无向连通图，边带长度和海拔。每次询问给出起点 v 和水位线 p ：海拔不超过 p 的边不可通车，其余边可通车。从 v 出发可通车到达任意点，然后可步行沿任意边走到节点 1，步行总长为经过边的长度之和。

每次给定 p, v 。求最小步行总长，并输出。

$n \leq 2 \times 10^5$, $m \leq 4 \times 10^5$, $Q \leq 4 \times 10^5$ 。

例题 F 题解

我们首先要知道可以坐车到达的点有哪些。根据上面的讨论，显然是最大生成树的 Kruskal 重构树中， v 祖先的子树。

于是我们这样做：Dijkstra 从 1 开始的最短路径 dis_i 。建立 Kruskal 重构树，然后对树上每个节点维护 mn_x 表示 x 子树内所有点的 $\min dis_i$ 。

每次询问时，倍增跳 v 的祖先，找到点权 $> p$ 的最浅祖先，答案就是这个点的 mn 。若不存在点权 $> p$ 的祖先，答案应为 dis_v 。

时间复杂度 $\mathcal{O}((m + q) \log m)$ 。

目录

- ① 生成树相关
- ② 树上差分
- ③ 树上启发式合并
- ④ 杂题

树上差分 - 树上前缀和

差分是前缀和的逆运算，我们先说树上前缀和。

前面说到，求 $s \rightarrow t$ 的距离可以用：

$$d_s + d_t - 2d_{lca}$$

表示，其中 d_x 表示从 x 到 1（根）的边权和。这实际上就是一种边权的前缀和。

树上差分 - 树上前缀和

那么，如果想表示 $s \rightarrow t$ 的点权和，设 $fsum_x$ 表示 x 到 1 的点权和，则有：

$$sum_{s \rightarrow t} = fsum_s + fsum_t - fsum_{lca} - fsum_{fa_{lca}}$$

(可以画图辅助理解)

如果要求子树和，我们前面说子树内 DFS 序为连续段，所以设 pre_x 为 DFS 序 $[1, x]$ 的点权和，则有：

$$sum_x = \sum_{dfn_x}^{dfn_x + sz_x - 1} = pre_{dfn_x + sz_x - 1} - pre_{dfn_x - 1}$$

其实就是序列前缀和。

实际上这是点权和，如果想求边权，可以用点表示其父边。

树上差分

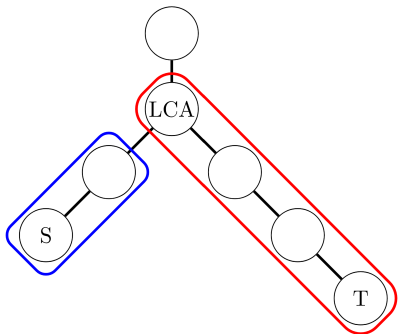


图: 树上点差分

我们利用子树和的逆运算表示差分。即，一个点的真实值为其子树内差分数组之和。

那么我们注意到，当为一个点的差分 $+1$ 时，它本身和它的祖先都会 $+1$ 。

由是，我们会对一条返祖的链做差分，如果 $x \rightarrow y$ 要 $+1$ ， y 为祖先，则 cf_x 要 $+1$ ， cf_{fa_y} 要 -1 。

拓展到一般的路径，如图，可以将路径分为两个返祖链。于是我们知道要

$$cf_s \leftarrow cf_s + v, cf_{lca(s,t)} \leftarrow cf_{lca(s,t)} - v,$$

$$cf_t \leftarrow cf_t + v, cf_{fa(lca(s,t))} \leftarrow cf_{fa(lca(s,t))} - v.$$

树上差分

如果是边差分，类似前面说到的技巧，将边的信息放在下面的节点中，于是 $s \rightarrow t$ 就变成了 $s \rightarrow t$ 除了 lca 以外的点。拆成的两个路径也就是上面的图不包括 LCA，由是：

$$cf_s \leftarrow cf_s + v,$$

$$cf_t \leftarrow cf_t + v,$$

$$cf_{lca(s,t)} \leftarrow cf_{lca(s,t)} - 2v.$$

例题 G

G - Max Flow P

给定一棵树，以及 K 条路径。每条路径将其经过的所有节点（含端点）的值增加 1。
求所有节点中值的最大值。

$n \leq 5 \times 10^4$, $K \leq 10^5$ 。

例题 G

G - Max Flow P

给定一棵树，以及 K 条路径。每条路径将其经过的所有节点（含端点）的值增加 1。
求所有节点中值的最大值。

$n \leq 5 \times 10^4$, $K \leq 10^5$ 。

模板题。

另外，由于有了树剖，似乎直接使用树上前缀和/差分就比较少了。但它现在常常作为转化的一步出现，后面会有例题；而且不管怎么说，它起码少一个 $\log$ 。

目录

- ① 生成树相关
- ② 树上差分
- ③ 树上启发式合并
- ④ 杂题

树上启发式合并

树上启发式合并通常处理离线问题，是优雅的暴力。

树上启发式合并

树上启发式合并通常处理离线问题，是优雅的暴力。

树上启发式合并有两种类型：

- 每个节点都维护一个独立的容器（e.g. map, vector），合并时“小的塞进大的”，把小节点的容器整个“拆了”，把里面的元素暴力插入到大节点的容器里，插完后子节点的容器就废弃掉。是比较直接的“启发式合并”。
- 全局通常只有一个容器，分轻重儿子遍历，先递归轻儿子然后清空容器，之后递归重儿子，不清空，最后非递归（for 循环）轻儿子子树来得到答案。

两种各有优劣，第一种更加直接，第二种能处理的问题更多，复杂度更优。一般说树上启发式合并指的是第二种，虽然它并不是合并，也有人叫它“链分治”。

第二种通常需要先学过树链剖分，这里我们就讲第一种。不妨我们就叫它“朴素树上启发式合并”。

例题 H

H - XOR Tree

n 个顶点的树，点有点权 a_x ，路径的权值定义为该路径上所有顶点的值的按位异或。我们称一棵树是“好”的，如果不存在权值为 0 的简单路径。最少修改多少个点的权值，使得树变为好树。

$$n \leq 2 \cdot 10^5$$

例题 H 题解

取 1 为根。

首先利用前缀“和”转化：设 $fsum_x$ 表示 $1 \rightarrow x$ 的路径异或和，路径 $u \rightarrow v$ 上的异或和等于 $fsum_u \oplus fsum_v \oplus a_{lca}$ 。

然后我们考虑这样一个贪心过程：从下到上依次考虑每一个点 x 作为 lca，如果这个点作为 lca 的时候有路径异或和为 0 的情况，那么就修改这个 lca 的权值为一个不可能与任何点异或和为 0 的值，例如 $2^{x+259675}$ 。

考虑这样做为什么是对的。如果我们令一个点的权值为上面的那个值，那么从这个点的子树内伸出来的所有路径都是不可能为 0 的。我们选取尽可能靠上的点，可以最大化这个效果。

例题 H 题解

我们对每个点开一个 set，记录这个子树内的点的 $fsum$ 值。合并时，设 S 是目前集合， T 是要并入的集合，则对 T 中每一个元素 t_i ，检查 S 中有没有等于 $t_i \oplus a_x$ 的元素。

- 如果有，说明存在一个 lca 为 x 的路径，其权值为 0。这时需要修改 x 的权值。修改之后，从 x 子树内伸出来的所有路径都不可能为 0，对外面没有贡献，直接清空 x 的 set 并结束 x 子树的全部操作。

使用启发式合并，时间复杂度 $\mathcal{O}(n \log^2 n)$ 。

例题 I

树上启发式合并有时用于部分分。

I - 人员调度 (36 分)

给定一棵有根树，根为 1。每个节点上初始有 k 个带正权员工。每个员工可被移动到其初始节点的任意后代节点（含自身）。每个节点最终最多贡献一个员工的权值（若该节点分配到至少一个员工，则贡献其中最大权值；否则贡献为 0）。目标是最大化所有节点贡献之和。

初始有一组员工，随后进行 m 次操作：增加一个员工（给定初始节点和权值），或删除一个已有员工。每次操作后输出当前最大总贡献。

$k \times m \leq 10^5$ ，权值 $\leq 10^5$ 。

例题 I 部分分题解

考虑这样的贪心过程：从下到上考虑，到一个点时，先去递归处理自己儿子子树的最优解，然后考虑这个点有的员工，如果能代替自己子树内的点使得贡献更大，也就是，目前子树内的最小贡献小于还没有分配的权值最大的员工，那就代替它。注意，这种代替可能是代替一个员工，也可能只是放到一个没有员工的地方。（没有员工相当于有一个权值为 0 的员工）

于是我们为每一个点开一个小根堆，存储其子树内的节点的贡献。到达 x 之后，依次合并每一个子树，小并大。全部并完之后，给这个点的初始员工从大到小排序，反复看能否替代堆顶直到不能替代。结束后，给这个点安排目前最大的员工，放进堆里。

时间复杂度 $\mathcal{O}(km \log^2 k)$ ，可以通过 36 pts。使用 pbds 可并堆实现一个 $\log$ ，多通过 12 分。

再看例题 B

B - Kay and Snowflake

给定一棵树，求出以每个点为根的子树的重心。

$$n \leq 3 \times 10^5$$

例题 B 另解

首先我们知道，删除一个点形成的连通块有两类，一类是自己儿子的子树，一类是除了自己子树以外的点。

设删去的点为 a ，目前求重心的是 x 子树。我们要保证第二类连通块不超过 $\frac{sz_x}{2}$ ，实际上就是要求 $sz_x \leq 2 \times sz_a$ 。我们希望第一类连通块的最大值不超过 $\frac{sz_x}{2}$ ，或者说，在保证第二类合法的情况下，第一类连通块尽可能小。

注意到第一类连通块大小不会随着 x 的变化而变化，因此，设这个值是 mx_a 。我们将 x 子树内的所有 mx_a 存入一个小根堆中，然后弹出 $2sz_a \leq sz_x$ 的点，此刻的堆顶就是 mx_a 最小且第二类连通块满足条件的点，即，重心。

弹出的这些点在未来也不会有用，所以可以使用朴素的树上启发式合并实现 $\mathcal{O}(n \log^2 n)$ ，或者使用可并堆实现 $\mathcal{O}(n \log n)$ 。

目录

- ① 生成树相关
- ② 树上差分
- ③ 树上启发式合并
- ④ 杂题

例题 J

J - 大融合

给定 n 个孤立点，初始无边。依次进行两种操作：

- 连接 x, y ，保证连接前它们不属于同一连通块。
- 查询边 (x, y) 的负载：定义为当前包含该边的树中，所有经过该边的简单路径数量。

规模： $n, Q \leq 10^5$ 。

例题 J 题解

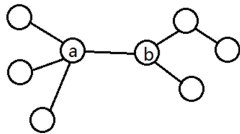
注意到每次查询点 a , b 两端构成的简单路径个数, 等价于原来的连通块删掉这个边之后两个连通块的大小之积。

直接维护整体不太容易, 于是考虑离线, 先把这棵最终的树 (可能为森林) 建出来, 同时随便选一个根, 每个点维护当前连通状态下子树大小。

假设 a 是 b 的父亲, 则 a 和 b 之间路径的答案为

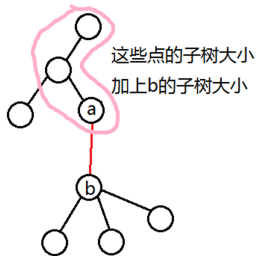
$(\text{当前连通块大小} - b \text{ 子树大小}) \times b \text{ 子树大小}$ 。

每个点维护子树大小, 用并查集维护每个连通块的大小。



例题 J 题解

则每次连接一条边的时候，等价于把一条链上的子树大小都加上一个值。比如连接 a 和 b 就是把： a 所在连通块里面深度最浅的那个点到 a 这一条链上的点加一个数。深度最浅的那个点用并查集维护即可。



问题转化为链加，单点求值。树上差分 + 树状数组维护即可，时间复杂度为 $\mathcal{O}(q \log n)$ 。

例题 K

K - 情报传递

给你一棵树，初始每个点没有点权，两种操作

1 x: 让一个点从当前时刻开始，每秒操作该点点权 $+1$

2 x y c: 查询一条链中有多少点的点权大于 c

其中每秒操作该点点权 $+1$ 就是指我每操作一次，无论是否和那个点有关，那个点权值都会 $+1$

1 操作对于每个点只会开始一次。

例题 K 题解

考虑我们肯定不能每次把 1 操作的点 $+1$ ，这样复杂度不对。

从 2 操作入手。查询 $x \rightarrow y$ 中有多少点权 $> c$ 的点。而点权是一秒 $+1$ 。所以等价于查 $x \rightarrow y$ 中有多少当前时间 - 开始时间 $> c$ 的点，即查询有多少点满足当前时间 $-c >$ 开始时间。相当于查询一条链中权值小于某个特定值的点的个数。这样可以树链剖分 + 树套树做到 $\mathcal{O}(m \log^3 n)$ 。

例题 K 题解

考虑我们肯定不能每次把 1 操作的点 $+1$ ，这样复杂度不对。

从 2 操作入手。查询 $x \rightarrow y$ 中有多少点权 $> c$ 的点。而点权是一秒 $+1$ 。所以等价于查 $x \rightarrow y$ 中有多少当前时间 - 开始时间 $> c$ 的点，即查询有多少点满足当前时间 $-c >$ 开始时间。相当于查询一条链中权值小于某个特定值的点的个数。这样可以树链剖分 + 树套树做到 $\mathcal{O}(m \log^3 n)$ 。

进一步研究。如果第 i 次操作是 1 x ，则相当于第 i 时刻 x 点开始。可以把操作 1 当成第 i 时刻的时候点 x 的权值从 0 变成了 1。如果第 i 次操作是 2 $x y c$ ，则相当于查询链 $x \rightarrow y$ 中开始时间 $< i - c$ 的点的个数，等价于查询第 $i - c$ 时刻链 $x \rightarrow y$ 上的和。

通过之前讲的树上差分 + 树状数组直接维护即可， $\mathcal{O}(m \log n)$ 。

例题 L

L - 小明的树

一棵以 1 为根的树， $2 \sim n$ 初始均未点亮。给出 $2 \sim n$ 的排列，按顺序点亮节点。1 号点自始至终不点亮。每点亮一个节点后，若当前亮灯集合满足“任意亮灯节点的子树内的点均全部点亮”，则称此时为有效时刻，并将当前亮灯集合在树中形成的连通块数量累加入答案；否则什么都不干，所有有效时刻的贡献总和即为该树的答案。支持 m 次操作：每次删一条边并加一条边，保证操作后仍是一棵树。每次操作后输出新树的答案（初始答案也需输出）。

规模： $n, m \leq 5 \times 10^5$ 。

例题 L 题解

需要用到两个 Trick。

Trick 1:

“点亮节点子树内的节点都是点亮的”等价于没有点亮的节点形成连通块。

这并不总是成立，当根节点可以被点亮时就不成立，但这个题并不需要考虑。考虑证明。其实这个结论和割点一些性质有点像（根节点特殊），所以从割点角度入手。一棵树（不考虑根节点）除了叶子节点外一定都是割点，那么选中就意味着树被分割，除非把它的子树也选上或者选上除了它子树以外的所有节点。所以上面结论充要性显然。

例题 L 题解

点边容斥

森林连通块数 = 点数 - 边数。

因此，设 i 步之后点亮的点的“导出森林”有 f_i 个连通块，未点亮的点有 g_i 个连通块，那么结果就是：

$$\sum_{i=1}^{n-1} [g_i = 1] \cdot f_i$$

而 $f_i = i - e\left(\bigcup_{j=1}^i a_j\right)$ ， $g_i = (n - i) - e\left(\bigcup_{j=i+1}^n a_j\right)$ 。加减边实际上是区间加减，维护答案即维护 g 最小时 f 的和。

一个块中 f 和 g 的 tag 相对独立，可以使用线段树。

时间复杂度： $\mathcal{O}((n + m) \log n)$ 。

* 例题 M

*M - minimax

n 个点的树，根为 1，每个点最多两个儿子。

定义节点 x 的权值为：

1. 若 x 没有子节点，权值会直接给出，保证这类点中每个点的权值互不相同。
2. 若 x 有子节点，它的权值有 p_x 的概率为子节点中权值最大值， $1 - p_x$ 的概率为子节点中权值最小值。

假设 1 号节点的权值有 m 种可能，其权值为第 i 小的概率为 D_i ，求 $D_{1 \sim m}$ 。

$n \leq 3 \times 10^5$ ，时间可以调大到 3s。

* 例题 M 题解

设 $f(x, val)$ 表示点 x 权值为 val 概率，如果只有一个儿子，可以直接继承。

如果有两个儿子，则需要合并答案。

记两个子树的答案为 $L(val)$ 和 $R(val)$ ，题目中说了一个权值只会出现一次，转移会稍微简单一点。

设 val 在左子树中出现

$$f(x, val) = L(val) \cdot \left(p_x \cdot \sum_{i=val+1}^n R(i) + (1 - p_x) \cdot \sum_{i=1}^{val-1} R(i) \right)$$

右子树类似，不再赘述。

* 例题 M 题解

注意到一个点的取值种类不会超过其子树大小，这启发我们用启发式合并或线段树合并。今天讲的是启发式合并，就讲讲这个的做法：

设左子树比较大，右子树比较小，所以我们先让父亲节点继承下来左子树。

对于右子树出现的值，我们直接暴力计算。关键是左子树。

* 例题 M 题解

我们再次观察这个式子：

$$f(x, val) = L(val) \cdot \left(p_x \cdot \sum_{i=val+1}^n R(i) + (1 - p_x) \cdot \sum_{i=1}^{val-1} R(i) \right)$$

注意到，父亲的答案相当于左子树继承的答案乘一个依赖于右子树的值，而比较好的是，乘的这个被右子树出现的权值分成一段一段，每段内部相同，这个段数和右子树大小一样。

因此，问题转化为暴力进行区间乘。

这样，总的复杂度 $\mathcal{O}(n \log^2 n)$ ，比较卡，毕竟当年是作为 60% 的做法的。

* 例题 M 题解

那么这个题的正解是什么呢？

考虑线段树合并。

还是开权值线段树，当走到一个节点时，如果 L 和 R 都有，那么就递归下去。

如果只有一棵树有，比如 L 有，类似上面的可以发现，对这个节点的所有值，乘上的都是同一个式子，打一下 `tag` 标记即可下传。

时间复杂度 $\mathcal{O}(n \log n)$ ，证明的话就是线段树合并，简单说：考虑动态开点，一个节点会开 $\log n$ 个节点，注意到因为我们不会深入，也就是说我们不会重造节点（或者说只会重造 $\mathcal{O}(\text{小集合})$ 个节点，启发式结论知道总的复杂度一个 $\log$ ），所以一个节点只会销毁一次。

闲话

闲话

注意到线段树合并、树上启发式合并、树上可并堆是有些相仿的，都是解决答案数不超过子树大小的这类问题。

其中，树上启发式合并泛用性最强，也最好写。虽然是双 $\log$ ，但常数小，其中一个 $\log$ 一般还卡不满。

我打比赛的时候所有线段树合并全部都用启发式合并，虽然偶尔会被制裁但是很少。